

Deep Networks

Regularization: Explicit and Implicit

Introduction to Deep Learning · Class 18

Outline

Why a Preference Is Unavoidable

Explicit Regularization

Implicit Regularization

Bias Built Into the Model

Putting It Together

Why a Preference Is Unavoidable

From measuring the error to choosing the solution

Class 17 ended with the fact that past the interpolation threshold many parameter vectors fit the data equally well, and the test error is decided by which one the procedure returns.

Settled in Class 17

error = bias² + variance + noise

many exact fits exist above the threshold

smooth interpolants generalize better

Open today

how to move the bias–variance balance on purpose

which of them the procedure returns, and why

what makes a procedure prefer one

Two mechanisms: a term we add to the objective, and a preference the optimiser already has without being asked.

Notation for today

$E(\mathbf{w})$ the training error of Class 13

$\Omega(\mathbf{w})$ the regularization term

λ, α its coefficient

$\tilde{E}(\mathbf{w})$ $E(\mathbf{w}) + \lambda \Omega(\mathbf{w})$

$\lambda_i, \mathbf{u}_i$ eigenvalues and eigenvectors of $\mathbf{H}$

η the step size, as in Class 13

$\tilde{L}$ the *modified* loss of an optimiser

B number of batches in an epoch

- Bishop writes the penalty coefficient λ and Prince writes it λ too; Bishop's α appears when the penalty is read as a prior precision
- $\mathbf{H}$ and its eigen-decomposition are the ones from Class 13

Bishop & Bishop (2024), Eq. 9.1; Prince (2023), Eq. 9.2.

Learning from data alone is not possible

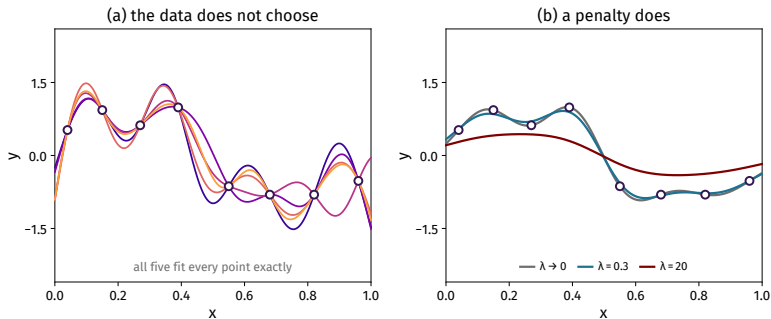
Proposition 1 (No free lunch)

Averaged over all possible problems, every learning algorithm performs equally well: one that beats the average on some problems must fall below it on others.

- So no algorithm is universally best, and the useful question is which problems an algorithm is **biased towards**
- The space of all problems includes relations no application would ever produce; real ones are mostly smooth, and most models are biased towards smoothness
- A finite parameter count is itself a bias; there is no unbiased option

Wolpert (1996); Bishop & Bishop (2024), §9.1.2.

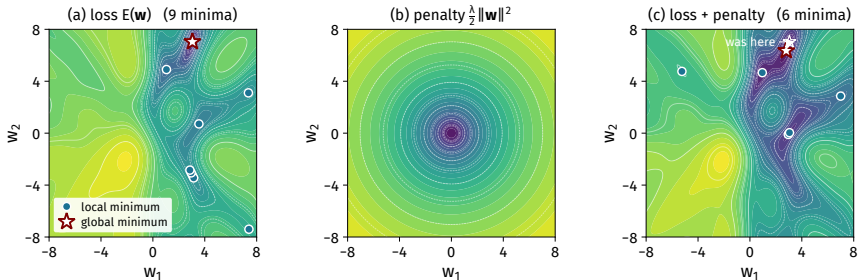
The data does not determine the function



(a) five functions, each through all eight observations: the training error cannot separate them. (b) a penalty on the weights singles one out, and its coefficient decides which.

Bishop & Bishop (2024), §9.1.1.

A penalty reshapes the whole surface



$\hat{y} = \sin(w_1 x) + \frac{1}{2} \sin(w_2 x)$ on 40 points. (a) the loss: nine local minima (dots), one global (star). (b) the penalty $\frac{\lambda}{2} \|\mathbf{w}\|^2$. (c) their sum: six minima remain, and the global one has moved towards the origin.

Three places to put a preference

Where	Example	Covered
the model class	basis functions, depth, weight sharing	today
the objective	$\lambda \ \mathbf{w}\ ^2$, the lasso	today
the algorithm	the step size, the batch size	today
the data	augmentation, added noise	Class 19
the stopping rule	early stopping	Class 19

The first three are properties of the model and the optimiser and apply before a single epoch is run. The last two are things done to the training procedure, and are the subject of the next class.

Explicit Regularization

The regularized objective

$$\hat{\mathbf{w}} = \arg \min_{\mathbf{w}} \left[\underbrace{\sum_n \ell_n(\mathbf{w})}_{\text{fit}} + \lambda \underbrace{\Omega(\mathbf{w})}_{\text{preference}} \right], \quad \Omega(\mathbf{w}) = \frac{1}{2} \|\mathbf{w}\|^2.$$

- Ω returns a larger value for parameters we like less; λ sets how much that opinion counts
- The minima of $\tilde{E}$ are not the minima of E , so training converges somewhere else on purpose
- For the quadratic choice the gradient is $\nabla \tilde{E} = \nabla E + \lambda \mathbf{w}$, which is why it is called **weight decay**: each step pulls $\mathbf{w}$ towards the origin

Bishop & Bishop (2024), Eqs. 9.1, 9.5; Prince (2023), Eq. 9.2.

The penalty is a prior

Proposition 2 (Maximum a posteriori)

Maximising the posterior $\prod_n p(Y_n | \mathbf{x}_n, \mathbf{w}) p(\mathbf{w})$ is the same as minimising $\sum_n \ell_n(\mathbf{w}) + \lambda \Omega(\mathbf{w})$ with

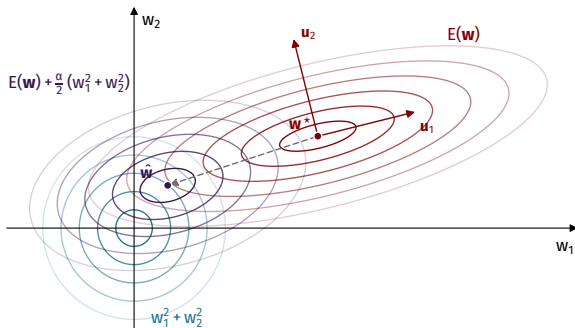
$$\lambda \Omega(\mathbf{w}) = -\log p(\mathbf{w}),$$

so a quadratic penalty is a zero-mean Gaussian prior of precision λ , and the lasso is a Laplace prior.

- Choosing a penalty is choosing what we believed before seeing any data
- The awkwardness is that the belief is expressed over *parameters*, while what we actually know concerns the *function*

Prince (2023), §9.1.1, Eqs. 9.3–9.4; Bishop & Bishop (2024), §9.2.

Weight decay, drawn

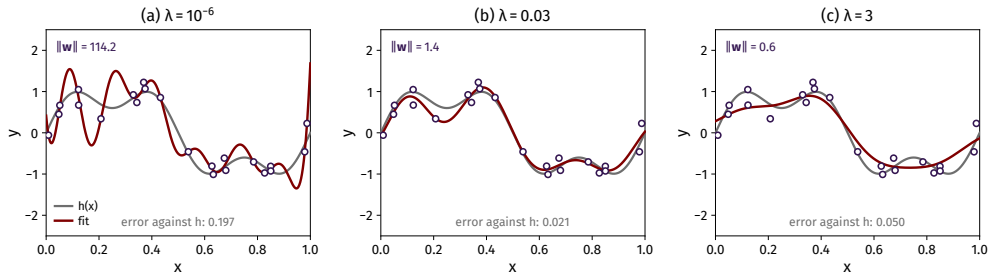


Contours of the error (maroon), the penalty (teal) and their sum (indigo), axes u_i of the Hessian drawn at w^* .

The minimum moves 2.32 along u_1 , where the error is insensitive, and 0.17 along u_2 .

Bishop & Bishop (2024), Fig. 9.3.

What the coefficient costs



The same 45-parameter model on the same 22 points. The weight norm falls from 114 to 0.6; the error against h falls from 0.197 to 0.021 and then rises to 0.050.

Prince (2023), Fig. 9.2.

A regularizer should respect the model's symmetries

Proposition 3 (Consistent regularizers)

Rescaling the inputs, $x_i \rightarrow ax_i + b$, leaves the network function unchanged provided the first-layer weights transform as $w_{ji} \rightarrow w_{ji}/a$, and similarly for the outputs. Simple weight decay is not invariant under this, but the layer-wise regularizer

$$\frac{\lambda_1}{2} \sum_{w \in \mathcal{W}_1} w^2 + \frac{\lambda_2}{2} \sum_{w \in \mathcal{W}_2} w^2, \quad \text{biases excluded,}$$

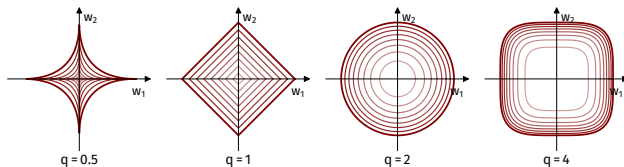
is, provided $\lambda_1 \rightarrow a^{1/2} \lambda_1$ and $\lambda_2 \rightarrow c^{-1/2} \lambda_2$.

- Otherwise changing the units of a feature would change which solution we prefer, which is not a preference about the world

Bishop & Bishop (2024), Eqs. 9.6–9.15.

Other shapes of penalty

$$\Omega(\mathbf{w}) = \frac{1}{2} \sum_{j=1}^W |w_j|^q, \quad \text{equivalently} \quad \min_{\mathbf{w}} E(\mathbf{w}) \quad \text{subject to} \quad \sum_{j=1}^W |w_j|^q \leq \eta.$$



Level sets of the penalty. Only $q \leq 1$ has corners on the axes; only $q \geq 1$ is convex; $q = 1$ alone has both, and is the lasso.

Bishop & Bishop (2024), Eqs. 9.18–9.20, Fig. 9.5.

Why the lasso sets coefficients to zero

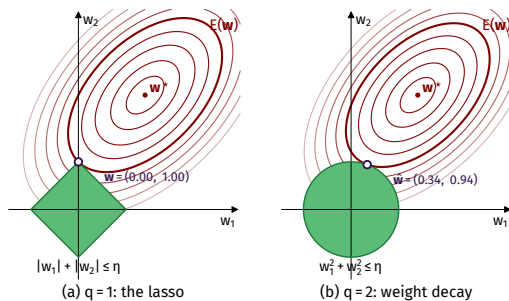
Proposition 4 (Sparsity from corners)

*In the constrained form the solution lies where a contour of E first touches the region $\sum_j |w_j|^q \leq \eta$. For $q = 1$ that region is a polytope whose vertices lie on the coordinate axes, so for a generic error function contact occurs at a vertex and some coordinates are **exactly** zero. For $q = 2$ the boundary is smooth and has no such points.*

- Increasing λ , equivalently shrinking η , drives more coefficients to zero and the corresponding basis functions out of the model
- Weight decay shrinks the same coefficients but never reaches zero

Tibshirani (1996); Bishop & Bishop (2024), §9.2.2, Fig. 9.6.

The two geometries side by side



Error contours (maroon) and constraint region (green), same E and same η . (a) the lasso makes contact at a vertex, so $\hat{w}_1 = 0$ exactly. (b) the quadratic penalty makes contact on a smooth arc and merely reduces it, to 0.34.

Bishop & Bishop (2024), Fig. 9.6.

Implicit Regularization

The optimiser has a preference too

- Nothing so far explains why an unregularized network, trained to zero training error, generalizes at all
- On a surface with a whole valley of global minima, the loss is equal everywhere along it, so **only the algorithm** can decide where to stop
- Gradient descent does not move neutrally: the discretisation itself expresses a preference, and so does the sampling of batches
- Both preferences can be written down as extra terms in a modified loss, and both can be checked

Prince (2023), §9.2.

What the step size is really minimising

Theorem 5 (Implicit regularization of gradient descent)

Continuous gradient flow follows $\partial \mathbf{w} / \partial t = -\nabla L(\mathbf{w})$. Discrete descent $\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \alpha \nabla L$ instead follows, to leading order in α , the flow of the *modified* loss

$$\tilde{L}_{\text{GD}}(\mathbf{w}) = L(\mathbf{w}) + \frac{\alpha}{4} \|\nabla L(\mathbf{w})\|^2.$$

- The extra term is a penalty on *steepness*: the trajectory is pushed away from places where the surface is sharp
- It vanishes at any stationary point, so it moves the path, not the set of minima
- Larger α means a stronger penalty — the step size is a regularization coefficient

Prince (2023), Eq. 9.8; Barrett & Dherin (2021).

Where the extra term comes from

- Expand one discrete step to second order: $w - \alpha \nabla L$ differs from the flow evaluated at time α by a term of order α^2 .
- Along the flow, ∇L itself changes at rate $-\mathbf{H}\nabla L$, so the flow's own second-order term carries $+\frac{\alpha^2}{2}\mathbf{H}\nabla L$.
- The discrete step has no such term, so the two disagree at order α^2 by exactly $\frac{\alpha^2}{2}\mathbf{H}\nabla L$.
- Ask which modified loss would reproduce that: we need $\nabla \tilde{L} = \nabla L + \frac{\alpha}{2}\mathbf{H}\nabla L$.
- That is a gradient, of $L + \frac{\alpha}{4}\|\nabla L\|^2$, since $\nabla \frac{1}{2}\|\nabla L\|^2 = \mathbf{H}\nabla L$. □

The penalty is not put there by anyone. It is the price of taking steps of finite length instead of following the curve.

Where the extra term comes from

- Expand one discrete step to second order: $\mathbf{w} - \alpha \nabla L$ differs from the flow evaluated at time α by a term of order α^2 .
- **Along the flow, ∇L itself changes at rate $-\mathbf{H}\nabla L$, so the flow's own second-order term carries $+\frac{\alpha^2}{2}\mathbf{H}\nabla L$.**
- The discrete step has no such term, so the two disagree at order α^2 by exactly $\frac{\alpha^2}{2}\mathbf{H}\nabla L$.
- Ask which modified loss would reproduce that: we need $\nabla \tilde{L} = \nabla L + \frac{\alpha}{2}\mathbf{H}\nabla L$.
- That is a gradient, of $L + \frac{\alpha}{4}\|\nabla L\|^2$, since $\nabla \frac{1}{2}\|\nabla L\|^2 = \mathbf{H}\nabla L$. □

The penalty is not put there by anyone. It is the price of taking steps of finite length instead of following the curve.

Where the extra term comes from

- Expand one discrete step to second order: $\mathbf{w} - \alpha \nabla L$ differs from the flow evaluated at time α by a term of order α^2 .
- Along the flow, ∇L itself changes at rate $-\mathbf{H}\nabla L$, so the flow's own second-order term carries $+\frac{\alpha^2}{2}\mathbf{H}\nabla L$.
- **The discrete step has no such term, so the two disagree at order α^2 by exactly $\frac{\alpha^2}{2}\mathbf{H}\nabla L$.**
- Ask which modified loss would reproduce that: we need $\nabla \tilde{L} = \nabla L + \frac{\alpha}{2}\mathbf{H}\nabla L$.
- That is a gradient, of $L + \frac{\alpha}{4}\|\nabla L\|^2$, since $\nabla \frac{1}{2}\|\nabla L\|^2 = \mathbf{H}\nabla L$. □

The penalty is not put there by anyone. It is the price of taking steps of finite length instead of following the curve.

Where the extra term comes from

- Expand one discrete step to second order: $\mathbf{w} - \alpha \nabla L$ differs from the flow evaluated at time α by a term of order α^2 .
- Along the flow, ∇L itself changes at rate $-\mathbf{H}\nabla L$, so the flow's own second-order term carries $+\frac{\alpha^2}{2}\mathbf{H}\nabla L$.
- The discrete step has no such term, so the two disagree at order α^2 by exactly $\frac{\alpha^2}{2}\mathbf{H}\nabla L$.
- **Ask which modified loss would reproduce that: we need $\nabla \tilde{L} = \nabla L + \frac{\alpha}{2}\mathbf{H}\nabla L$.**
- That is a gradient, of $L + \frac{\alpha}{4}\|\nabla L\|^2$, since $\nabla \frac{1}{2}\|\nabla L\|^2 = \mathbf{H}\nabla L$. □

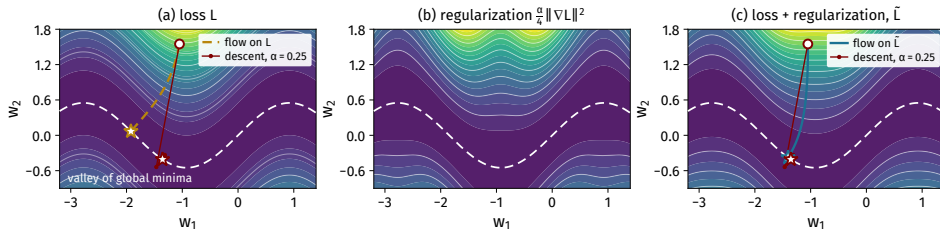
The penalty is not put there by anyone. It is the price of taking steps of finite length instead of following the curve.

Where the extra term comes from

- Expand one discrete step to second order: $\mathbf{w} - \alpha \nabla L$ differs from the flow evaluated at time α by a term of order α^2 .
- Along the flow, ∇L itself changes at rate $-\mathbf{H}\nabla L$, so the flow's own second-order term carries $+\frac{\alpha^2}{2}\mathbf{H}\nabla L$.
- The discrete step has no such term, so the two disagree at order α^2 by exactly $\frac{\alpha^2}{2}\mathbf{H}\nabla L$.
- Ask which modified loss would reproduce that: we need $\nabla \tilde{L} = \nabla L + \frac{\alpha}{2}\mathbf{H}\nabla L$.
- **That is a gradient, of $L + \frac{\alpha}{4}\|\nabla L\|^2$, since $\nabla \frac{1}{2}\|\nabla L\|^2 = \mathbf{H}\nabla L$.** □

The penalty is not put there by anyone. It is the price of taking steps of finite length instead of following the curve.

Testing the claim



Every point of the dashed valley is a global minimum. (a) the flow on L and discrete descent with $\alpha = 0.25$ reach different points of it. (b) the penalty on the squared gradient, zero on the floor. (c) with it added, the flow arrives where descent did; its endpoint error falls like $\alpha^{1.9}$, the plain flow's like $\alpha^{1.0}$.

Prince (2023), Fig. 9.3.

What sampling batches adds

Theorem 6 (Implicit regularization of SGD)

Averaging over the possible orderings of B batches, stochastic gradient descent follows the flow of

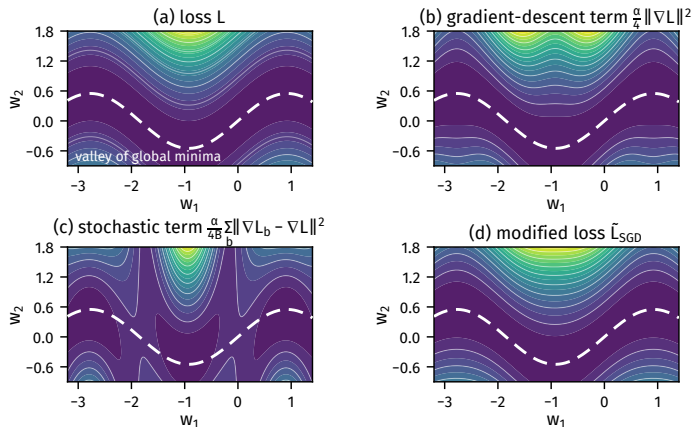
$$\tilde{L}_{\text{SGD}} = L + \frac{\alpha}{4} \|\nabla L\|^2 + \frac{\alpha}{4B} \sum_{b=1}^B \|\nabla L_b - \nabla L\|^2,$$

*where L_b is the loss on batch b . The extra term is the **variance of the batch gradients**.*

- So SGD prefers places where the batches *agree* about which way is downhill
- The term scales as $1/B$: smaller batches regularize harder
- Like the first term it vanishes where all gradients vanish

Prince (2023), Eq. 9.9; Smith et al. (2021).

The two implicit penalties, measured



(a) the loss. (b) the gradient-descent term, penalising steepness. (c) the stochastic term, penalising disagreement between the $B = 12$ batches. (d) their sum, the modified loss. Both terms vanish on the valley floor.

Prince (2023), Fig. 9.4.

What this predicts, and what is observed

Change	Effect on the penalty	Observed
larger step size α	both terms grow, linearly in α	full-batch descent generalizes better
smaller batch size	variance term grows as $1/B$	small batches generalize better
momentum, adaptive rates	change the modified loss again	the analysis has to be redone

- These are consistent with the analysis but not proved by it: the same observations admit other explanations, such as noise helping the iterate leave sharp regions

Prince (2023), §9.2.2 and Fig. 9.5.

Bias Built Into the Model

Invariance and equivariance

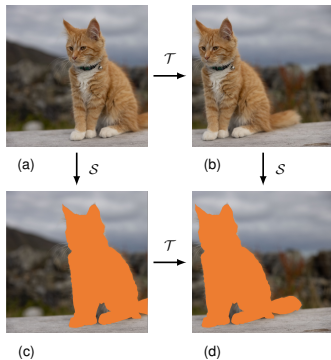
Definition 7 (Invariance and equivariance)

Let $\mathcal{T}$ be a transformation of the input. A map $\mathcal{C}$ is *invariant* under $\mathcal{T}$ if $\mathcal{C}(\mathcal{T}(I)) = \mathcal{C}(I)$, and a map $\mathcal{S}$ is *equivariant* if $\mathcal{S}(\mathcal{T}(I)) = \tilde{\mathcal{T}}(\mathcal{S}(I))$ for a corresponding transformation $\tilde{\mathcal{T}}$ of the output. Invariance is the case $\tilde{\mathcal{T}} = \text{id}$.

- Classify an object: translate the image, the label is unchanged — invariance
- Segment an object: translate the image, the segmentation translates with it — equivariance
- The transformations leaving a property unchanged form a **group**: closed, associative, with identity and inverses

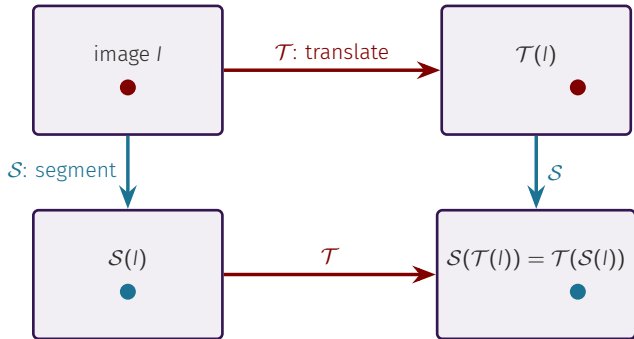
Bishop & Bishop (2024), Eqs. 9.2–9.4, §9.1.3–9.1.4.

Equivariance, on an image



Translating the image and then segmenting gives the same result as segmenting and then translating: the diagram commutes. Bishop & Bishop (2024), Fig. 9.2.

Equivariance, drawn



either route
gives the same
result

- Building the symmetry into the architecture makes the diagram commute by construction, rather than hoping training discovers it

Bishop & Bishop (2024), Fig. 9.2.

Four ways to obtain an invariance

Approach	How	Cost
pre-processing	compute invariant features first	may discard useful information
a penalty	punish output change under $\mathcal{T}$	only small transformations
augmentation	train on transformed copies	more data, more compute (<i>Class 19</i>)
architecture	make the symmetry structural	must be known in advance

- Learning the invariance from raw data instead is usually hopeless: a shift of a few pixels changes every pixel value, and the invariances compose combinatorially

Bishop & Bishop (2024), §9.1.3; Simard et al. (1992).

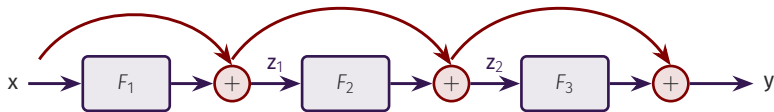
Parameter sharing: a hard constraint

- A penalty makes some weights small. **Parameter sharing** instead forces groups of weights to be equal, so there are fewer degrees of freedom than connections
- The shared value is still learned; what is fixed is that the group moves together
- Backpropagation needs only a small change: the gradient of a shared value is the sum of the gradients of its copies, and this is how translation equivariance enters a convolutional network
- **Soft** weight sharing replaces the hard constraint with a mixture-of-Gaussians prior, so the groups, their centres and spreads are all learned

Bishop & Bishop (2024), §9.4–9.4.1; Nowlan & Hinton (1992).

Residual connections

$$z_1 = F_1(x) + x, \quad z_2 = F_2(z_1) + z_1, \quad y = F_3(z_2) + z_2.$$



- Each block learns the **residual** $F_l(z_{l-1}) = z_l - z_{l-1}$, so producing the identity needs only small parameters
- The bias is towards shallow behaviour, with depth available when the data pays for it

Bishop & Bishop (2024), Eqs. 9.35–9.38; He et al. (2015).

Why residual connections smooth the surface

Proposition 8 (A residual network is an ensemble of shallower ones)

Substituting the intermediate variables gives

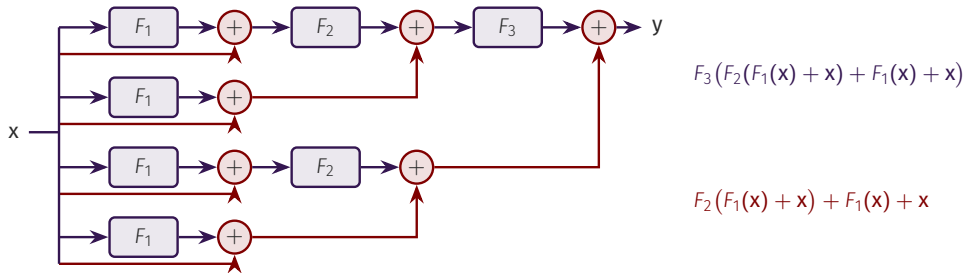
$$\mathbf{y} = F_3(F_2(F_1(\mathbf{x}) + \mathbf{x}) + F_1(\mathbf{x}) + \mathbf{x}) + F_2(F_1(\mathbf{x}) + \mathbf{x}) + F_1(\mathbf{x}) + \mathbf{x},$$

a sum over every subset of the blocks: paths of depth 3, 2, 1 and 0 all in parallel.

- The deep path is still there, so nothing is lost in representational terms
- But the shallow paths have well-behaved gradients, which is why the error surface is far less shattered than a plain network of the same depth

Bishop & Bishop (2024), Eq. 9.40, Figs. 9.12, 9.15; Balduzzi et al. (2017).

The same network, expanded



The three-block network written out. Every route from x to y is a network in its own right, from three blocks deep to none; the two expressions are the arguments of the final addition.

Bishop & Bishop (2024), Fig. 9.15.

Putting It Together

What each mechanism prefers

Mechanism	Prefers	Controlled by
L^2 / weight decay	small weights, smooth functions	λ
lasso, $q = 1$	few non-zero weights	λ
layer-wise penalty	the same, respecting rescaling	λ_1, λ_2
gradient descent	flat regions of the surface	step size α
stochastic descent	regions where batches agree	batch size B
parameter sharing	functions with the chosen symmetry	the grouping
residual connections	shallow behaviour unless paid for	the architecture

The first three are written into the objective and can be switched off. The next two are properties of the optimiser and cannot.

Takeaways

1. Finite data never determines a function, so every learning algorithm must prefer some solutions; there is no unbiased option (Proposition 1)
2. A penalty on the parameters is a prior over them (Proposition 2)
3. Weight decay suppresses the directions the error function is insensitive to, which are the ones fitting noise, and leaves the rest nearly alone
4. A penalty should respect the symmetries of the model, which simple weight decay does not (Proposition 3)
5. $q = 1$ is the only exponent that is both convex and has corners, which is why the lasso is sparse (Proposition 4)
6. Gradient descent implicitly minimises $L + \frac{\alpha}{4} \|\nabla L\|^2$, so the step size is a regularization coefficient (Theorem 5)
7. Stochastic descent adds the variance of the batch gradients, scaling as $1/B$ (Theorem 6)
8. Symmetry, parameter sharing and residual connections put the preference in the architecture instead

Class 19 turns to what we do to the procedure and the data: early stopping, ensembles, dropout and added noise.

References i

1. C. M. Bishop & H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024. §9.1 (Eqs. 9.2–9.4, Figs. 9.1–9.2); §9.2 (Eqs. 9.1, 9.5–9.20, Figs. 9.3–9.6); §9.4; §9.5 (Eqs. 9.32–9.41, Figs. 9.12–9.16).
2. S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023. §9.1 (Eqs. 9.1–9.5, Figs. 9.1–9.2); §9.2 (Eqs. 9.6–9.10, Figs. 9.3–9.5).
3. D. H. Wolpert, “The lack of a priori distinctions between learning algorithms”, *Neural Computation* 8(7), 1996.
4. R. Tibshirani, “Regression shrinkage and selection via the lasso”, *J. Royal Statistical Society B* 58(1), 1996.
5. P. Simard, B. Victorri, Y. LeCun & J. Denker, “Tangent prop”, NIPS 1992.
6. S. Nowlan & G. E. Hinton, “Simplifying neural networks by soft weight sharing”, *Neural Computation* 4(4), 1992.
7. D. G. T. Barrett & B. Dherin, “Implicit gradient regularization”, ICLR 2021.
8. S. L. Smith, B. Dherin, D. G. T. Barrett & S. De, “On the origin of implicit regularization in stochastic gradient descent”, ICLR 2021.
9. K. He, X. Zhang, S. Ren & J. Sun, “Deep residual learning for image recognition”, CVPR 2016 (arXiv 2015).

References ii

- 10. D. Balduzzi et al., “The shattered gradients problem”, ICML 2017.
- 11. M. M. Bronstein, J. Bruna, T. Cohen & P. Veličković, “Geometric deep learning”, arXiv:2104.13478, 2021.

Questions?